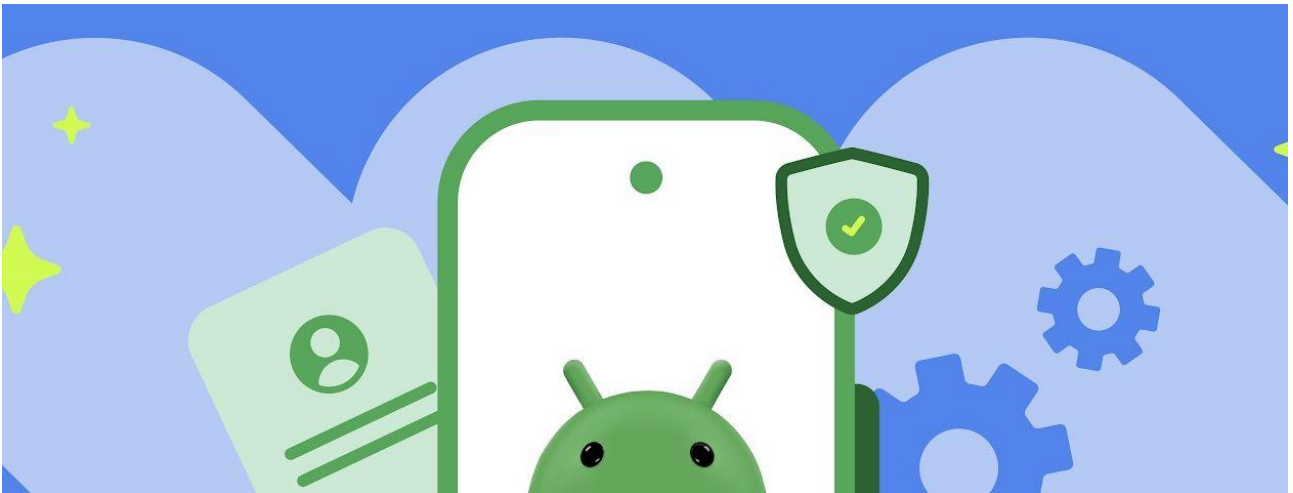




Université de Dschang - Faculté des Sciences

TP - INDIVIDUEL

PROGRAMMATION MOBILE

CINÉTRACK

INF 392 — Edition 2025/2026

Prof Bomgni Alain

Année Académique 2025-2026

"Derrière chaque tap sur un écran, il y a des milliers de lignes de code écrites par quelqu'un qui a refusé de dire 'c'est impossible'."

Cahier des Charges – **CineTrack** (nom provisoire, chaque étudiant renomme selon sa thématique)

Ce projet compte XX% de votre note finale de Contrôle Continu.

Date de rendu : 24 mai

Présentation du projet

Vous développez une application Android native de gestion de films ou séries centrée sur **une seule thématique de votre choix** : romance, action, anime, horreur, science-fiction, films africains, bollywood, séries policières, comédies... peu importe, mais une seule, du début à la fin.

L'application utilise l'API gratuite **TMDB (The Movie Database)** pour rechercher des films réels, afficher leurs affiches, synopsis et informations. L'utilisateur constitue ensuite sa collection personnelle en sauvegardant les films qui l'intéressent localement sur son téléphone.

Le nom de l'app, les couleurs, les polices et l'identité visuelle doivent refléter votre thématique et votre personnalité. Une app sur les animes ne ressemble pas à une app sur les films de romance. [Nb : *TMDB est un élément vraiment obligatoire pour ce projet*]

L'API TMDB – ce que vous devez savoir

TMDB est une base de données collaborative de films et séries, entièrement gratuite et ouverte aux développeurs. Elle fournit une API REST qui retourne des données en JSON.

Pour l'utiliser vous devez créer un compte sur themoviedb.org, demander une clé API gratuite dans les paramètres de votre compte, et inclure cette clé dans chaque requête que votre application envoie.

Exemples de requêtes utiles pour votre projet :

Rechercher un film par titre :

https://api.themoviedb.org/3/search/movie?api_key=VOTRE_CLE&query=inception&language=fr-FR

Récupérer les films populaires d'un genre :

https://api.themoviedb.org/3/discover/movie?api_key=VOTRE_CLE&with_genres=28&language=fr-FR (28 = action, 10749 = romance, 16 = animation...)

Afficher une affiche : les URLs d'images se construisent ainsi :

https://image.tmdb.org/t/p/w500/CHEMIN_IMAGE

La documentation complète est sur developers.themoviedb.org. Vous devrez la lire et comprendre comment parser les réponses JSON que l'API retourne.

Fonctionnement concret de l'application

Quand l'utilisateur ouvre l'app, il voit sa collection personnelle de films déjà sauvegardés. Il peut rechercher un nouveau film via la barre de recherche l'app envoie une requête à

TMDB, reçoit une liste de résultats avec affiches et infos, et les affiche. L'utilisateur choisit un film, consulte sa fiche détaillée récupérée depuis TMDB, et peut l'ajouter à sa collection avec un statut personnel (à voir, en cours, vu) et une note personnelle sur 5.

Il y a donc deux types de données dans votre app. Les données TMDB qui viennent d'internet à chaque recherche et ne sont pas modifiables. Et les données personnelles de l'utilisateur statut, note, avis écrit, date d'ajout qui sont sauvegardées localement sur le téléphone et persistent entre les sessions.

Concepts obligatoires à couvrir

Widgets : TextView, EditText, Button, ImageView pour les affiches, CheckBox, Switch, RadioButton/RadioGroup, Spinner, DatePicker ou DatePickerDialog, RatingBar pour la note personnelle.

Layouts : LinearLayout, ScrollView, TableLayout ou GridLayout pour la fiche détail, RelativeLayout ou ConstraintLayout.

Listes : RecyclerView avec CardView et adaptateur personnalisé pour afficher les résultats de recherche et la collection.

Menus : Toolbar avec SearchView fonctionnelle connectée à TMDB, menu d'options, menu contextuel sur les cartes de la collection.

Feedback : Toast et AlertDialog.

Réseau et JSON : requête HTTP vers TMDB dans un thread en arrière-plan (Executors ou AsyncTask), parsing de la réponse JSON avec JSONObject et JSONArray, chargement des affiches depuis une URL avec la bibliothèque Glide ou Picasso, gestion du cas sans connexion avec un message approprié.

Cycle de vie : onCreate, onPause, onResume, onSaveInstanceState et onRestoreInstanceState présents et commentés avec explication de vos choix.

Persistence : SharedPreferences pour les paramètres utilisateur et la clé API, fichiers internes pour la collection personnelle de films.

Navigation : au moins trois activités avec passage de données via Intent.

Structure minimale des écrans

Écran 1 – Splash : logo, nom de l'app thématique, chargement automatique.

Écran 2 – Ma collection : Toolbar avec titre et menu, RecyclerView avec CardView affichant les films sauvegardés avec leur affiche, titre, note personnelle et statut. FloatingActionButton pour aller à la recherche. Menu contextuel sur chaque carte : modifier le statut, modifier la note, supprimer.

Écran 3 – Recherche TMDB : barre de recherche connectée à l'API, RecyclerView des résultats avec affiches récupérées depuis TMDB, clic sur un résultat pour voir le détail.

Écran 4 – Détail d'un film : affiche en grand, titre, synopsis, note TMDB, date de sortie récupérés depuis l'API. Section personnelle avec Spinner pour le statut, RatingBar pour la

note personnelle, EditText pour l'avis écrit, DatePicker pour la date de visionnage. Bouton Ajouter à ma collection.

Écran 5 — Paramètres : nom de l'utilisateur, thématique choisie, sauvegardés via SharedPreferences.

Ce qu'il faut rendre

Vous rendez une archive .zip du dossier complet du projet Android Studio tel qu'il est sur votre machine — dossier app/, build.gradle, AndroidManifest.xml, tous les dossiers res/ et java/. Le correcteur doit pouvoir ouvrir le projet dans Android Studio et le lancer sans modification.

Attention : ne mettez pas votre clé API TMDb en dur dans le code versionné (règle de sécurité). Placez-la dans un fichier local.properties ou dans les SharedPreferences et expliquez dans votre rapport comment la configurer pour que le correcteur puisse tester l'app.

En plus du .zip vous rendez un **mini-rapport PDF de 5 pages maximum** contenant : la thématique choisie et sa justification, la description de vos écrans, vos choix techniques expliqués concept par concept, et les difficultés rencontrées avec comment vous les avez résolues.

Vous incluez dans le projet un fichier **SOURCES.md** qui liste toutes vos sources — documentation TMDb, documentation Android, tutoriels, Stack Overflow — avec pour chaque source la notion que vous cherchiez à comprendre. Ce fichier est obligatoire.

Tout est déposé sur la plateforme indiquée par l'enseignant avant le **24 mai à 23h59**.

Évaluation — 20 points

Code — 10 points : fonctionnalités présentes et fonctionnelles, couverture des concepts obligatoires, appel API TMDb opérationnel, qualité du code, commentaires, gestion des erreurs réseau, persistance locale fonctionnelle. [

Modalités de dépôt

Vous devez déposer votre travail via le lien qui vous sera communiqué par l'enseignant. Le dépôt comprend trois éléments obligatoires : l'archive .zip du projet Android Studio, une vidéo de démonstration de l'application d'une durée maximale de 5 et d'une taille maximale de 30 Mo Max, et le fichier SOURCES.md. Ces trois éléments doivent être déposés ensemble avant le **24 mai à 00h00**. Tout dépôt incomplet ou en retard, même d'une minute, ne sera pas accepté => une note de 0/10 pour le code.]

Oral — 10 points : vous présentez votre mini-rapport lors de la présentation Orale et répondez aux questions sur votre code. L'enseignant peut interroger sur n'importe quelle ligne, méthode ou choix. Si vous avez écrit le code vous-même et que vous le comprenez, l'oral se passe bien **FOR SURE**.

Avertissement sur l'intelligence artificielle

On se connaît : L'utilisation excessive de l'intelligence artificielle est formellement proscrite. Vous pouvez consulter de la documentation, des tutoriels, des exemples c'est normal. Mais vous devez comprendre et être capable d'expliquer chaque ligne que vous écrivez.

Le fichier SOURCES.md est là pour montrer votre démarche de recherche personnelle. L'oral est conçu précisément pour détecter le code non compris. Un étudiant qui ne peut pas expliquer son propre code à l'oral sera sanctionné indépendamment de la qualité apparente du code rendu. Cherchez, essayez, faites des erreurs, corrigez-les. C'est exactement comme ça qu'on apprend à développer.

Université de Dschang – Département Maths-Info – Licence 3 – Semestre 6